Verzweigungen

Programme sollen nicht immer dieselben Anweisungen ausführen. Mit `if` und `else` können sie abhängig von einer Bedingung unterschiedliche Wege einschlagen. Du lernst, solche Verzweigungen zu lesen und selbst zu programmieren.

Im vorherigen Kapitel haben wir Bedingungen kennengelernt. Eine Bedingung ist entweder `True` oder `False`.

Nun verwenden wir diesen Wahrheitswert, um den **Ablauf eines Programms zu steuern**. Ein Programm kann abhängig von einer Bedingung unterschiedliche Wege einschlagen. Dies nennt man eine **Verzweigung**.

1. Einfache Verzweigung mit `if`

Wir wollen ein Programm schreiben, das ein Quadrat mit Seitenlänge `x` zeichnet. Falls `x` negativ oder null ist, soll nicht gezeichnet werden.

```
import turtle

t = turtle.Turtle()
x = int(input("Gib eine Seitenlänge ein: "))

if x > 0:
    for i in range(4):
        t.forward(x)
        t.right(90)
```

Ist die Bedingung `True`, werden die eingerückten Anweisungen ausgeführt. Ist sie `False`, werden sie übersprungen. Allgemein sieht eine einfache `if`-Bedingung wie folgt aus:

```
if bedingung:
    anweisung(en)
```

!!! note “Merke” Nach der Bedingung steht ein Doppelpunkt. Alle Anweisungen, die zur `if`-Anweisung gehören, werden **eingerückt**. Verwende für die Einrückung die Tabulator-Taste, damit alle gleichmässig eingerückt sind.

2. Zwei Fälle mit `else`

```
if x > 0:
    for i in range(4):
        t.forward(x)
        t.right(90)
else:
    print("Der eingegebene Wert ist ungültig.")
```

Der `else`-Teil wird ausgeführt, wenn die Bedingung des zugehörigen `if`-Teil zu `False` auswertet. Bei dieser Form wird genau **einer der beiden Wege** ausgeführt.

3. Mehrere Fälle mit `elif`

```
farbe = input("Gib eine Ampelfarbe ein: ")

if farbe == "red":
    print("Bleibe stehen.")
elif farbe == "orange":
    print("Bleibe stehen oder beeile dich.")
elif farbe == "green":
    print("Du kannst laufen.")
else:
    print("Diese Farbe gibt es bei einer Ampel nicht.")
```

Es können mehrere `elif`-Teile folgen. Die Bedingungen werden **von oben nach unten** überprüft. Sobald eine Bedingung wahr ist, wird der zugehörige Teil ausgeführt und die folgenden Teile werden übersprungen. Der `else`-Teil ist optional und steht am Schluss. Er wird dann ausgeführt, wenn keine der `if`-/`elif`-Teile zu `True` ausgewertet wurde.

4. Verknüpfte Bedingungen in Verzweigungen

Die logischen Operatoren `and`, `or` und `not` können direkt in einer Verzweigung verwendet werden.

Beispiel 1:

```
alter = int(input("Wie alt bist du? "))

if alter ⩾ 13 and alter ⩽ 19:
    print("Du bist ein Teenager.")
```

Beispiel 2:

```
farbe = input("Farbe: ")

if farbe == "red" or farbe == "orange":
    print("Nicht loslaufen.")
else:
    print("Du kannst laufen.")
```

5. Verschachtelte Verzweigungen

Eine `if`-Anweisung kann selbst wieder eine `if`-Anweisung enthalten.

```
if A:
    if B:
        print("Treffer")
```

Dies ist in diesem Fall gleichbedeutend mit:

```
if A and B:
    print("Treffer")
```

Auch

```
if A:
    print("Treffer")
```

```
elif B:
    print("Treffer")
```

kann – wenn in beiden Fällen dieselben Anweisungen ausgeführt werden – als `if A or B:` formuliert werden.

Nicht jede Verschachtelung lässt sich jedoch einfach durch `and` oder `or` ersetzen.

6. Die Reihenfolge von Bedingungen

```
alter = 15

if alter ⩾ 6:
    print("Primarschule")
elif alter ⩾ 13:
    print("Sekundarstufe")
```

Obwohl `alter ⩾ 13` wahr ist, wird `"Sekundarstufe"` nicht ausgegeben. Bereits die erste Bedingung ist wahr, deshalb wird der folgende `elif`-Teil übersprungen.

Eine mögliche Korrektur ist:

```
if alter ⩾ 13:
    print("Sekundarstufe")
elif alter ⩾ 6:
    print("Primarschule")
else:
    print("Kindergarten")
```

!!! note “Merke” Bei `if` – `elif` – `else` wird der **erste passende Fall** ausgeführt. Deshalb kann die Reihenfolge der Bedingungen den Programmablauf verändern.

Aufgaben

Die folgenden Aufgaben werden **ohne Computer auf Papier** gelöst.

Aufgabe 1 – Programmablauf nachvollziehen

```
x = 7

if x > 10:
    print("A")
elif x > 5:
    print("B")
else:
    print("C")

print("D")
```

a) Welche Ausgabe erzeugt das Programm?
b) Welche Ausgabe entsteht für `x = 12` ?

c) Welche Ausgabe entsteht für `x = 5`?

d) Erkläre, weshalb für `x = 12` nicht auch `"B"` ausgegeben wird.

Aufgabe 2 – Verzweigung schreiben

Ein Programm erhält eine ganze Zahl `x`. Ist `x` positiv, soll `"positiv"` ausgegeben werden; bei 0 `"0"`; sonst `"negativ"`. Schreibe die passende Verzweigung mit `if`, `elif` und `else`.

Aufgabe 3 – Bedingungen einsetzen

Ein Eintritt ist erlaubt, wenn eine Person mindestens 12 Jahre alt ist **und** ein gültiges Ticket besitzt. Die Variablen heissen `alter` und `ticket`; `ticket` kann den Wert `True` oder `False` haben.

a) Schreibe eine Verzweigung, die `"Eintritt erlaubt"` oder `"Eintritt nicht erlaubt"` ausgibt.

b) Welche Ausgabe entsteht für `alter = 15` und `ticket = False`?

c) Welche Ausgabe entsteht für `alter = 12` und `ticket = True`?

Aufgabe 4 – Programme vergleichen

Programm A

```
if x > 0:
    if x < 10:
        print("Treffer")
```

Programm B

```
if x > 0 and x < 10:
    print("Treffer")
```

a) Sind A und B für alle Werte von `x` äquivalent? Begründe.

Programm C

```
if x > 0:
    print("Treffer")
elif x < 10:
    print("Treffer")
```

Programm D

```
if x > 0 and x < 10:
    print("Treffer")
```

b) Sind C und D äquivalent?

c) Falls nicht: Finde mindestens einen Wert von `x`, bei dem sie sich unterschiedlich verhalten.

Aufgabe 5 – Fehler finden und korrigieren

Ein Programm soll folgende Einteilung vornehmen: unter 6 Jahren `"Kindergarten"`, von 6 bis und mit 12 Jahren `"Primarschule"`, ab 13 Jahren `"Sekundarstufe"`.

```
if alter ⩾ 6:
    print("Primarschule")
elif alter ⩾ 13:
    print("Sekundarstufe")
```

```
else:
    print("Kindergarten")
```

a) Erkläre, weshalb das Programm nicht korrekt funktioniert.
b) Gib einen konkreten Wert an, bei dem eine falsche Ausgabe entsteht.
c) Korrigiere das Programm.
d) Erkläre, weshalb die Reihenfolge der Bedingungen wichtig ist.

Aufgabe 6 – Verschachtelung umformen

```
if A:
    if B:
        print("X")
    else:
        print("Y")
else:
    print("Y")
```

Schreibe ein äquivalentes Programm, das **keine verschachtelte Verzweigung** verwendet. Verwende dafür eine verknüpfte Bedingung sowie `if` und `else`. Begründe kurz, weshalb dein Programm für alle Kombinationen von `A` und `B` dieselbe Ausgabe erzeugt.